Università degli Studi di Milano - Bicocca
Dipartimento di Informatica, Sistemistica e Comunicazione
Corso di Laurea in Informatica

Progetto di Metodi Informatici per la Gestione Aziendale

Sviluppo di diverse tipologie di Recommendation System

DAVIDE DONEDDU 879365

Contents

Introduzione **4**
0.1 Obiettivi del Progetto 4
0.1.1 Progetto Base 4
0.1.2 Progetto Intermedio 4
0.1.3 Progetto Avanzato 4
0.2 Dataset Utilizzato 4
0.3 Metodologia e Strumenti 5

1 Esplorazione e Pulizia dei Dataset **6**
1.1 Introduzione 6
1.2 Analisi del Dataset Rewiew 6
1.2.1 Struttura del Dataset 6
1.2.2 Esplorazione del Dataset 7
1.3 Pulizia Preliminare e Analisi della Sparsità 8
1.3.1 Pulizia Preliminare dei Dati 9
1.3.2 Analisi dei Duplicati 9
1.3.3 Analisi Sparsità della Matrice Originale 9
1.3.4 Filtraggio di Utenti e Prodotti 10
1.3.5 Analisi della Matrice Filtrata 11
1.3.6 Analisi della Matrice di Correlazione 12
1.4 Analisi del Dataset dei Metadata 13
1.4.1 Struttura del Dataset 13
1.4.2 Qualità dei Dati 13
1.4.3 Analisi delle Componenti Testuali 14
1.4.4 Riduzione ai Prodotti Presenti nel Dataset delle Recensioni 14

2 Recommendation System Based on Collaborative Filtering **16**
2.1 Introduzione 16
2.2 Preparazione della Matrice Utente–Prodotto 16
2.2.1 Creazione della Matrice 16
2.2.2 Train–Test Split 16
2.2.3 Indicizzazione 17
2.2.4 Costruzione della Matrice Utente–Prodotto 17
2.3 Ricerca della Configurazione Ottimale del Modello K-NN 17
2.3.1 Implementazione del Modello 17
2.3.2 Grid Search dei Parametri 17
2.3.3 Risultati Sperimentali 18
2.3.4 Configurazione Ottimale 19

2.4 Filling della Matrice di Rating 19
2.4.1 Predizione dei Valori Mancanti 19
2.4.2 Risultato del Filling 19
2.5 Segmentazione degli Utenti 19
2.5.1 Preprocessing e Riduzione della Dimensionalità 19
2.5.2 Scelta del Numero di Cluster 20
2.5.3 Risultato del Clustering 20
2.5.4 Visualizzazione dei Cluster 21
2.6 Generazione delle Raccomandazioni Top-N 21
2.6.1 Predizione Personalizzata 21
2.6.2 Raccomandazioni Basate sui Cluster 22
2.7 Matrix Factorization (SVD) e Confronto Finale 22
2.7.1 Implementazione della SVD 22
2.7.2 Confronto delle Performance 22
2.7.3 Analisi delle Raccomandazioni 23
2.8 Conclusioni 23

3 Recommendation System Content-Based 24
3.1 Introduzione 24
3.2 Preparazione dei Testi 24
3.2.1 Creazione del Campo Testuale 24
3.2.2 Pulizia e Preprocessing 24
3.2.3 Analisi Statistica dei Testi 25
3.3 Embedding Basati su Frequenza 26
3.3.1 TF-IDF 26
3.3.2 Similarità tra Item 26
3.4 Embedding Basati su Transformers 27
3.4.1 Modello Utilizzato 27
3.4.2 Estrazione degli Embedding 27
3.4.3 Similarità Item–Item 27
3.5 Predizione dei Rating con K-NN Content-Based 27
3.5.1 Definizione del Modello 27
3.5.2 Procedura di Predizione 28
3.5.3 Metriche di Valutazione 28
3.6 Confronto tra Tecniche di Embedding 28
3.6.1 TF-IDF: Analisi e Caratteristiche 28
3.6.2 TF-IDF vs Transformers 29
3.7 Confronto Finale: Content-Based vs Collaborative Filtering . . . 29
3.7.1 Performance Predittiva 29
3.7.2 Analisi Interpretativa 29
3.7.3 Conclusioni del Confronto 30

4 Progetto Avanzato: Sentiment Analysis sulle Review 31
4.1 Obiettivo del Progetto Avanzato 31
4.2 Dati Utilizzati 31
4.3 Fasi del Progetto Avanzato 31
4.3.1 Preprocessing Testuale delle Review 32
4.3.2 Creazione degli Embedding Testuali 32
4.3.3 Modellazione del Sentiment 32
4.4 Metriche di Valutazione 33

4.5 Risultati e Considerazioni 33

5 Conclusioni **35**
5.1 Sintesi del Lavoro Svolto 35
5.2 Risultati Principali 35
5.2.1 Collaborative Filtering 35
5.2.2 Content-Based Filtering 35
5.2.3 Sentiment Analysis 36
5.3 Confronto tra Modelli 36
5.4 Limitazioni del Lavoro 36
5.5 Conclusione Finale 36

Introduzione

0.1 Obiettivi del Progetto

L'obiettivo del progetto è stato quello di analizzare il dataset fornito di recensioni amazon al fine di implementare valutare e confrontare diversi tipi di Recommendation Systems.

Il progetto è stato diviso in tre livelli di complessità progressiva, ognuno esplorando una diversa tecnica di analisi.

- ### 0.1.1 Progetto Base

  Il progetto base si è concentrato sull'analisi esplorativa dei dataset e sul *collaborative filtering*, sistema di raccomandazione che analizza i dati di interazione utente-prodotto con l'obiettivo di fornire suggerimenti personalizzati all'utente.

- ### 0.1.2 Progetto Intermedio

  Il progetto intermedio ha integrato questo approccio con il *content-based filtering*, implementando sistemi di NLP per estrapolare le caratteristiche testuali dei prodotti per suggerire articoli affini a quelli già apprezzati.

- ### 0.1.3 Progetto Avanzato

  Infine, il progetto avanzato ha aggiunto una componente di *sentiment analysis*, che ha permesso di classificare le recensioni in base al sentimento espresso (positivo, negativo, neutro) per ottenere una comprensione ancora più profonda del feedback degli utenti.

0.2 Dataset Utilizzato

L'analisi è stata condotta sul set di dati **Amazon Reviews 2023**, specificamente sulla categoria "Amazon Fashion", che ha fornito una base di dati significativa per eseguire analisi complete e per testare l'efficacia dei diversi modelli.

0.3 Metodologia e Strumenti

Il flusso di lavoro ha consistito in una fase di esplorazione pulizia e preparazione del dataset, una seconda fase di implementazione dei vari sistemi con i loro requisiti e infine un lavoro di comparazione e stesura dei risultati ottenuti.

Per lo sviluppo del progetto sono state utilizzate diverse librerie Python, tra cui:

- **pandas** e **NumPy** per la manipolazione e l'analisi numerica dei dati;
- **scikit-learn** per l'implementazione di algoritmi di *machine learning*;
- **NLTK** per il *pre-processing* del linguaggio naturale;
- **Hugging Face** per l'utilizzo di modelli neurali avanzati.

Chapter 1

Esplorazione e Pulizia dei Dataset

1.1 Introduzione

Come visto nell'introduzione, per questo progetto sono stati usati due dataset legati provenienti da Amazon Reviews Dataset — categoria *Amazon Fashion*, versione disponibile su HuggingFace. La tabella seguente riassume le dimensioni del corpus originale:

Categoria	**#Utenti**	**#Prodotti**	**#Recensioni**	**Dataset**
Amazon_Fashion	2.0M	825.9K	2.5M	review, meta

Table 1.1: Statistiche di base del dataset Amazon Fashion (HuggingFace).

I dati sono suddivisi in due file principali:

- **Review Dataset**: contiene le recensioni degli utenti sui prodotti.
- **Metadata Dataset**: contiene le informazioni testuali e descrittive dei prodotti.

1.2 Analisi del Dataset Rewiew

1.2.1 Struttura del Dataset

Il dataset delle recensioni contiene un totale di 2.500.939 record, ogni record identifica una singola recensione lasciata da un utente un un prodotto specifico. Nel complesso, il dataset include:

- **2.035.490 utenti unici**, identificati tramite `user_id`;
- **874.297 prodotti unici** identificati tramite `asin`;
- **825.869 prodotti principali** (`parent_asin`), utilizzati per la fusione con i metadata;

- **10 attributi per ciascuna recensione**, che descrivono testo, punteggio, struttura del prodotto e informazioni sull'utente.

La Tabella 1.2 riporta nel dettaglio i campi presenti nel dataset delle recensioni e il loro significato (contenuto).

Field	**Type**	**Description**
rating	int64	Punteggio assegnato dall'utente al prodotto (1–5).
title	object	Titolo della recensione scritto dall'utente.
text	object	Descrizione completa della recensione scritta dall'utente.
images	object	Lista di immagini caricate dall'utente (molti record vuoti).
asin	object	Identificativo del prodotto specifico (in caso di varianti dello stesso) (ASIN).
parent_asin	object	Identificativo del prodotto principale genitore di tutte le sue varianti, usato come chiave per il dataset dei prodotti.
user_id	object	Identificativo univoco dell'utente che ha scritto la recensione.
timestamp	datetime64[ns]	Data e ora in cui la recensione è stata pubblicata.
helpful_vote	int64	Numero di voti "utile" ricevuti dalla recensione.
verified_purchase	bool	Indica se l'acquisto è verificato.

Table 1.2: Descrizione dei campi del dataset delle recensioni Amazon Fashion.

1.2.2 Esplorazione del Dataset

La struttura dei dati, e l'eventuale presenza di row non valide come record di intestazione o fine del dataset, è stata verificata tramite la stampa a schermo delle prime righe (`head`), delle ultime (`tail`) e di un campione casuale di 10 (`sample`). La tabella 1.3 riporta i campi contenuti nel dataset.

Rating	**Parent ASIN**	**User ID**
5	B0722QNRPW	AE22236AFRRSMQIKGG7TPTB75QEA
2	B010AN1JAG	AE2224EKBYOTIAXL7PC7KRQIXHOA
5	B07C9Z1FTH	AE222GY2VH7MEWI4N5AHPXB5CP5A
3	B008PWO57O	AE222H3FGXWLHRFUMGMS2RR57NDQ
4	B00925F8E2	AE222H3FGXWLHRFUMGMS2RR57NDQ

Table 1.3: Primi 5 record del dataset ridotto a rating, parent_asin e user_id.

Successivamente tramite il metodo `describe()` del dataframe ho ottenuto e analizzato le statistiche descrittive delle variabili numeriche: `rating`, `timestamp` e `helpful_vote`. Essendo unicamente utile in questo progetto il campo di rating, qui sotto si trova l'analisi della distribuzione dei suoi rating.

Analisi del campo `rating`

Il campo `rating` contiene i punteggi assegnati dagli utenti ai prodotti, con valori compresi tra 1 e 5.

Statistic	Rating
count	2,474,375
mean	3.969501
std	1.426897
min	1.000000
25%	3.000000
50%	5.000000
75%	5.000000
max	5.000000

Table 1.4: Statistiche descrittive del campo `rating` nel dataset delle recensioni.

L'analisi statistica evidenzia quanto segue:

- il numero di osservazioni coincide con il numero totale di righe del dataset (2.500.939);
- la media è pari a 3.96, mentre la mediana è 5. questo mostra l'alta inclinazione verso il massimo (perfettamente coerente con la situazione reale delle recensioni).
- i quartili mostrano che il 25% dei rating è pari a 3, mentre il 50% e il 75% sono pari a 5: oltre metà delle recensioni assegna il punteggio massimo (conferma la tendenza mostrata dalla media);
- la deviazione standard è pari a 1.42, valore elevato rispetto all'intervallo possibile (1–5), confermando un'ampia dispersione attorno alla media.

1.3 Pulizia Preliminare e Analisi della Sparsità

La fase iniziale del processo di data cleaning ha previsto diverse operazioni mirate a garantire coerenza, riduzione del rumore e qualità della matrice utente–prodotto finale. Le principali attività svolte sono state:

- rimozione di valori nulli e tuple duplicate;
- aggregazione delle recensioni duplicate (`user_id`, `parent_asin`) mantenendo il rating medio e il timestamp più recente;
- selezione delle colonne rilevanti (`user_id`, `parent_asin`, `rating`);
- analisi della distribuzione delle interazioni per utenti e prodotti;
- analisi della sparsità della matrice utente–prodotto.

1.3.1 Pulizia Preliminare dei Dati

Il workflow seguito durante questa fase di analisi sui dati è stato pensato in modo da migliorare la qualità dei dati e garantire sia una rappresentazione coerente del dataset originale ma alleggerendo il carico alla computazioni necessarie nel progetto. Le attività svolte comprendono:

- rimozione di valori nulli e tuple duplicate;
- aggregazione delle recensioni duplicate (`user_id`, `parent_asin`) tramite media del rating e selezione del timestamp più recente;
- selezione delle colonne rilevanti (`user_id`, `parent_asin`, `rating`);
- analisi statistica preliminare delle interazioni;
- valutazione della struttura della matrice utente–prodotto.

1.3.2 Analisi dei Duplicati

Il dataset originale conteneva un numero non trascurabile di duplicati, come mostrato in seguito:

$$\text{Duplicati} = 49{,}249 \qquad (1.97\% \text{ del totale})$$

Questi duplicati sono stati correttamente aggregati. La Tabella 1.5 riassume l'effetto del processo.

Table 1.5: Statistiche della fase di deduplicazione

Parametro	**Valore**
Recensioni iniziali	2,500,939
Recensioni dopo deduplicazione	2,474,375
Duplicati rimossi	26,564

1.3.3 Analisi Sparsità della Matrice Originale

Il dataset iniziale presenta una matrice utente–prodotto estremamente sparsa (come si evincere dal numero di utenti e di prodotti), caratteristica tipica dei sistemi di raccomandazione. Utilizziamo la definizione standard di sparsità:

$$\text{Sparsità} = 1 - \frac{\text{Numero di rating osservati}}{\text{Numero di possibili accoppiamenti utente–prodotto}}$$

La Figura 1.1 mostra un campione di 89 utenti e 100 prodotti, da cui emerge chiaramente che la matrice contiene quasi esclusivamente celle vuote. La sparsità del dataset originale risulta quindi **prossima al 100%**.

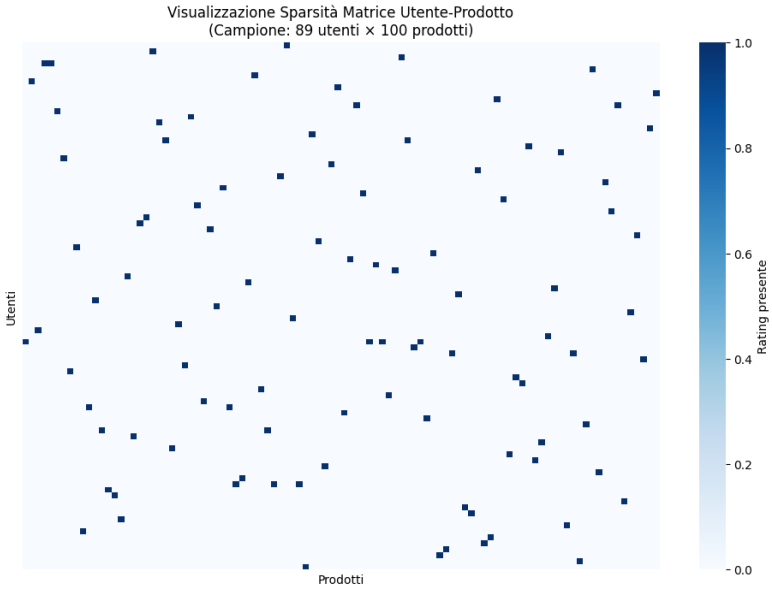


Figure 1.1: Matrice utente–prodotto originale (campione 89 × 100). La quasi totalità delle celle è priva di rating, indicando una sparsità prossima al 100%.

1.3.4 Filtraggio di Utenti e Prodotti

La matrice originale ha sparsità troppo alta, rendendo difficile agli algoritmi trovare pattern chiari e risultati coerenti. Per ottenere una struttura più densa e informativa è stato applicato un filtraggio iterativo basato sulle interazioni minime:

- almeno 3 interazioni per utente;
- almeno 5 interazioni per prodotto.

Tale processo elimina nodi rari fino al raggiungimento di una configurazione stabile.

La Tabella 1.6 riassume la struttura del dataset finale ottenuto dopo il filtraggio.

Table 1.6: Statistiche della matrice utente–prodotto dopo il filtraggio

Parametro	**Valore**
Utenti	967
Prodotti	572
Rating osservati	4,849
Dimensione matrice	553,124
Densità	0.8767%
Sparsità	99.12%

1.3.5 Analisi della Matrice Filtrata

Sparsità dopo il Filtraggio

Dopo la rimozione di utenti e prodotti con poche interazioni, la matrice rimane fortemente sparsa ma presenta un incremento significativo della densità:

$$\text{Sparsità}_{\text{filtrata}} = 1 - \frac{4849}{967 \times 572} = 0.9912 \quad (99.12\%)$$

La matrice ottenuta è sufficientemente densa per supportare i metodi di raccomandazione analizzati nei capitoli successivi.

Distribuzione delle Interazioni per Utente

- Media: 5.01 recensioni
- Mediana: 4
- Deviazione standard: 4.03

Table 1.7: Distribuzione delle recensioni per utente

Range	**Utenti**	**%**
3–5	731	75.6%
6–10	179	18.5%
11–20	49	5.1%
21–50	7	0.7%
≥51	1	0.1%

Distribuzione delle Interazioni per Prodotto

- Media: 8.48 recensioni
- Mediana: 7
- Deviazione standard: 6.09

Table 1.8: Distribuzione delle recensioni per prodotto

Range	**Prodotti**	**%**
3–5	158	27.6%
6–10	315	55.1%
11–20	79	13.8%
21–50	17	3.0%
51–100	3	0.5%
≥101	0	0.0%

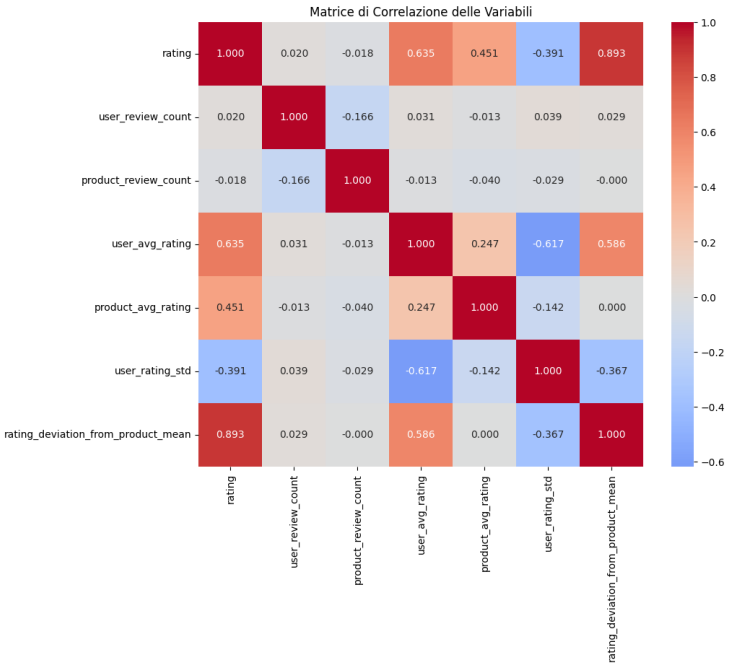


Figure 1.2: Matrice di correlazione tra rating, statistiche utente e statistiche prodotto.

1.3.6 Analisi della Matrice di Correlazione

Per valutare le relazioni tra le principali variabili ricavate dal dataset filtrato è stata calcolata una matrice di correlazione. Le variabili includono statistiche sugli utenti, sui prodotti e misure di deviazione dei rating.

Correlazioni più rilevanti

Le relazioni più significative osservate sono:

- **rating user_avg_rating (0.536)** Gli utenti mostrano coerenza nel proprio comportamento: chi tende ad assegnare valutazioni elevate mantiene tale tendenza.
- **rating product_avg_rating (0.558)** I prodotti generalmente apprezzati ricevono con maggiore probabilità recensioni elevate.
- **rating user_rating_std (-0.334)** Una maggiore variabilità nelle valutazioni di un utente è associata a rating mediamente più bassi.
- **rating rating_deviation_from_product_mean (0.830)** Correlazione molto alta, poiché la variabile misura direttamente lo scostamento del rating rispetto alla media del prodotto.

Sintesi delle evidenze

Dall'analisi emerge che:

- le medie dei rating degli utenti e dei prodotti sono i fattori più influenti nel determinare il valore della singola recensione;
- la variabilità dei rating dell'utente contribuisce a caratterizzare la coerenza del suo comportamento valutativo;
- il numero di recensioni per utente o prodotto non mostra alcuna relazione significativa con il rating;
- la deviazione dalla media del prodotto è una misura utile per individuare recensioni atipiche.

Questi elementi guidano la selezione delle feature da utilizzare nei modelli predittivi successivi.

1.4 Analisi del Dataset dei Metadata

1.4.1 Struttura del Dataset

Il dataset dei metadata è composto da **826.108 prodotti** descritti da **14 attributi**. I principali campi includono informazioni testuali, numeriche e categoriali, tra cui:

- `main_category`, `categories`, `store`;
- `title`, `description`, `features`;
- `average_rating`, `rating_number`, `price`;
- `images`, `videos`;
- `parent_asin`, utilizzato come chiave di collegamento con il dataset delle recensioni.

Il campo `bought_together` risulta completamente nullo e non è stato considerato nelle analisi successive.

1.4.2 Qualità dei Dati

L'analisi dei valori mancanti evidenzia una buona qualità complessiva del dataset. La maggior parte delle variabili risulta completa, con le seguenti eccezioni principali:

- il campo `price` presenta una percentuale di valori mancanti pari a circa **94%**;
- il campo `store` presenta circa **3.25%** di valori nulli;
- il campo `bought_together` è nullo per il **100%** delle istanze.

Tutti gli altri attributi risultano privi di valori mancanti. Nosostante questo ai fini del progetto intermedio sono stati utilizzati solo i campi **parent_asin**, **title**, **description**.

1.4.3 Analisi delle Componenti Testuali

Per le componenti testuali principali sono state analizzate le lunghezze in termini di numero di parole.

Titoli La lunghezza dei titoli risulta mediamente contenuta:

- lunghezza media: **14.3 parole**;
- mediana: **14 parole**;
- distribuzione concentrata tra 11 e 17 parole.

Descrizioni Le descrizioni mostrano una distribuzione fortemente sbilanciata:

- lunghezza media: **6.0 parole**;
- mediana pari a **0 parole**;
- solo **59.289 prodotti** su 826.108 presentano una descrizione non vuota.

Questa evidenza indica che la maggior parte dei prodotti non dispone di una descrizione testuale significativa.

1.4.4 Riduzione ai Prodotti Presenti nel Dataset delle Recensioni

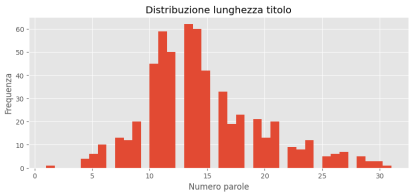
Al fine di garantire coerenza tra le analisi sui metadata e quelle basate sulle recensioni, il dataset è stato filtrato mantenendo esclusivamente i prodotti effettivamente presenti nel dataset ridotto delle review, utilizzando il campo `parent_asin` come chiave di collegamento.

Questa operazione ha portato alla costruzione del sottoinsieme finale `metadata_subset`, composto da **572 prodotti**, rappresentativi dell'intersezione tra le due sorgenti informative.

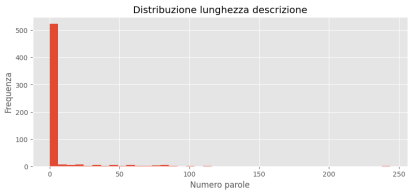
L'analisi statistica del sottoinsieme evidenzia che:

- i titoli presentano una lunghezza media di circa **14.5 parole**, con una distribuzione relativamente concentrata;
- le descrizioni risultano mediamente molto brevi (**4.2 parole**) e frequentemente assenti;
- il prezzo medio osservato è di circa **26.9**.

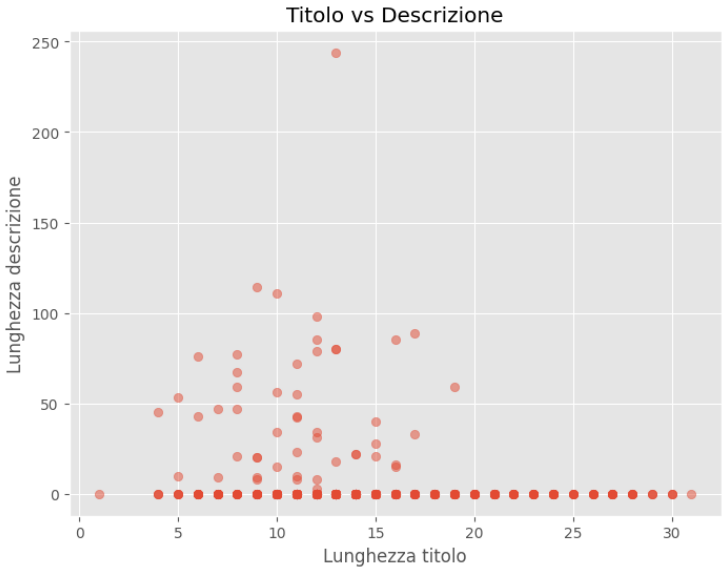
La Figura 1.3 mostra la distribuzione delle lunghezze dei titoli, delle descrizioni e la relazione tra le due variabili. Si osserva come le descrizioni siano spesso assenti o estremamente brevi, indipendentemente dalla lunghezza del titolo, indicando una limitata ricchezza informativa del contenuto testuale.



(a) Distribuzione lunghezza titoli



(b) Distribuzione lunghezza descrizioni



(c) Titolo vs descrizione

Figure 1.3: Analisi delle componenti testuali del dataset dei metadata filtrato

La riduzione al sottoinsieme finale consente di:

- eliminare prodotti privi di interazioni utente;
- migliorare l'efficacia delle tecniche di embedding testuale;
- garantire coerenza dati tra approcci di collaborative filtering e content-based filtering, con finalità di confronto tra Progetto Base e Intermedio.

Chapter 2

Recommendation System Based on Collaborative Filtering

2.1 Introduzione

L'obiettivo del progetto base è la realizzazione di un sistema di raccomandazione basato sul *Collaborative Filtering* (CF), utilizzando esclusivamente i rating del dataset delle recensioni degli utenti. Il modello sfrutta le interazioni utente–prodotto per stimare i rating mancanti e produrre raccomandazioni personalizzate.

Il lavoro è stato svolto su un sottoinsieme del dataset Amazon Fashion, filtrato, come visto nel primo capitolo, in modo da mantenere solo utenti e prodotti con un numero minimo di interazioni.

2.2 Preparazione della Matrice Utente–Prodotto

2.2.1 Creazione della Matrice

A partire dal dataset filtrato è stata costruita una matrice utente–prodotto $U \times I$, in cui ogni cella $M_{u,i}$ contiene il rating assegnato dall'utente u al prodotto i (inserendo 0 nel caso in cui la recensione non fosse presente). Le interazioni non osservate sono rappresentate tramite una struttura *sparse* nel formato *Compressed Sparse Row* (CSR), per ottimizzare memoria ed efficienza nelle fasi di addestramento.

2.2.2 Train–Test Split

Il dataset è stato suddiviso in train (80%) e test (20%) tramite `train_test_split`. Per evitare problemi di *cold start*, il mapping tra utenti/prodotti e indici interi è stato definito esclusivamente sul train set. Le interazioni del test set contenenti utenti o prodotti non presenti nel train sono state rimosse, per un totale di 12 righe eliminate.

2.2.3 Indicizzazione

Gli identificativi originali (`user_id`, `parent_asin`) sono stati convertiti in indici interi consecutivi, ottenendo:

$$\text{Utenti} = 0 \ldots 962, \qquad \text{Prodotti} = 0 \ldots 571$$

Tale indicizzazione permette di costruire in modo efficiente la matrice utente–prodotto utilizzata nei modelli di raccomandazione.

2.2.4 Costruzione della Matrice Utente–Prodotto

La matrice finale è stata costruita utilizzando la rappresentazione `coo_matrix`, successivamente convertita nel formato CSR per migliorare l'efficienza nelle operazioni di accesso e moltiplicazione.

$$M \in \mathbb{R}^{U \times I}, \qquad M_{u,i} = \begin{cases} \text{rating assegnato} & \text{se l'interazione esiste} \\ 0 & \text{altrimenti (assenza di rating)} \end{cases}$$

La matrice risultante costituisce la base per gli algoritmi di raccomandazione presentati nel capitolo successivo.

2.3 Ricerca della Configurazione Ottimale del Modello K-NN

2.3.1 Implementazione del Modello

Il modello K-NN è stato implementato manualmente replicando il comportamento del `KNNBaseline` di Surprise. L'implementazione comprende:

- calcolo delle componenti di baseline (μ, b_u, b_i) tramite ALS;
- calcolo delle similarità tra utenti o prodotti (cosine, Pearson);
- applicazione dello shrinkage;
- modalità *user-based* e *item-based*;
- predizione tramite K vicini pesati dalla similarità.

2.3.2 Grid Search dei Parametri

La ricerca della configurazione migliore è stata condotta esplorando le seguenti combinazioni:

- tipo di similarità: cosine, Pearson;
- shrinkage: 10, 50, 100;
- modalità: user-based e item-based;
- valori di k: 5, 10, 20, 30, 40, 60, 80, 100.

Per ogni configurazione sono state calcolate MSE, RMSE sul test set.

2.3.3 Risultati Sperimentali

I risultati mostrano un comportamento molto stabile per il modello item-based, con RMSE sempre intorno a 1.10 indipendentemente da k o dallo shrinkage. Al contrario, il modello user-based presenta prestazioni significativamente superiori.

Risultati per similarità `cosine`

La configurazione user-based con shrinkage 50 ha prodotto i risultati migliori:

$$\text{RMSE}_{\text{best}} = 1.06257, \qquad k_{\text{best}} = 10.$$

I principali trend osservati sono:

- per valori piccoli di k, l'RMSE decresce rapidamente fino a $k = 10$;
- oltre $k = 20$, l'errore tende a stabilizzarsi o peggiorare leggermente;
- lo shrinkage influisce marginalmente, ma valori intermedi (50) migliorano la stabilità.

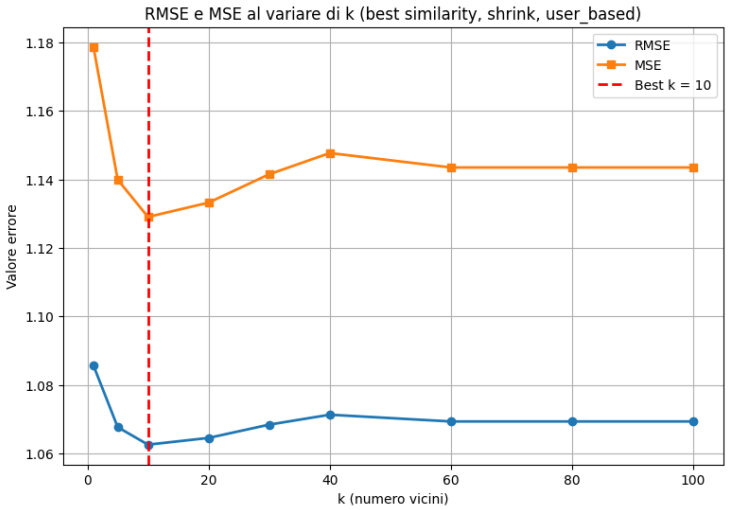


Figure 2.1: Andamento di RMSE e MSE al variare di k (similarità cosine, shrinkage ottimale).

Confronto tra Similarità

Dalla sperimentazione emerge che:

- **cosine** offre sistematicamente le migliori prestazioni;
- **Pearson** produce RMSE compresi tra 1.09 e 1.12, inferiori rispetto a cosine;
- il modello *item-based* non raggiunge mai prestazioni competitive rispetto allo *user-based*.

2.3.4 Configurazione Ottimale

La migliore configurazione complessiva trovata dalla grid search è:

similarità:	cosine,
shrinkage:	50,
modalità:	user-based,
numero di vicini:	$k = 10$.

Con le seguenti metriche finali:

$$\text{RMSE} = 1.06257, \qquad \text{MSE} = 1.12906,$$

Questi risultati mostrano che l'uso di similarità cosine in modalità user-based, combinato con uno shrinkage medio, consente di ottenere il miglior compromesso tra bias di base e capacità predittiva dei vicini.

2.4 Filling della Matrice di Rating

2.4.1 Predizione dei Valori Mancanti

A partire dalla configurazione ottimale individuata per il modello K-NN (cosine similarity, user-based, shrinkage 50, $k = 10$), la matrice utente–prodotto è stata completata stimando tutti i valori mancanti tramite predizione in stile *KNNBaseline*. Il modello utilizza i vicini più simili per ricostruire i rating non osservati, ottenendo così una rappresentazione completamente densa della matrice.

2.4.2 Risultato del Filling

La matrice originale di dimensione 963×572 è stata mantenuta nel suo formato, ma tutte le celle prive di rating sono state riempite tramite predizione. Nel complesso, il processo ha stimato e inserito:

$$\mathbf{546\,957} \quad \text{valori mancanti}$$

ottenendo così una matrice densa utilizzabile per i modelli successivi.

2.5 Segmentazione degli Utenti

2.5.1 Preprocessing e Riduzione della Dimensionalità

Per applicare tecniche di clustering sulla matrice densa è stata effettuata una riduzione della dimensionalità con PCA. Per il clustering sono state mantenute le prime 20 componenti principali, mentre la visualizzazione è stata realizzata in 2D tramite le prime due componenti.

2.5.2 Scelta del Numero di Cluster

La selezione del valore ottimale di k è stata condotta tramite:

- **Elbow Method**, che mostra una riduzione significativa dell'inertia fino a $k \approx 6$;
- **Silhouette Score**, che presenta un massimo locale per $k = 6$.

Sulla base di entrambi i criteri è stato scelto $k = 6$.

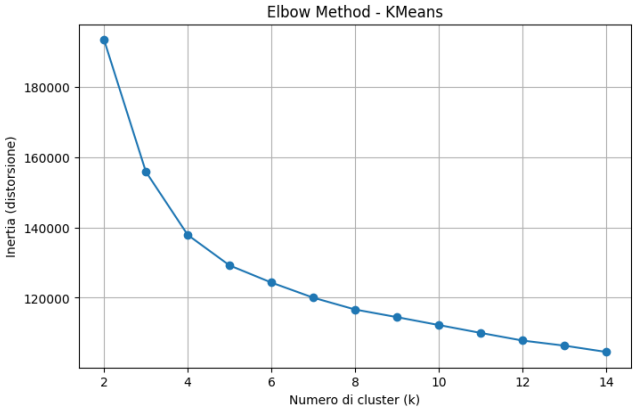


Figure 2.2: Elbow Method per la scelta del numero di cluster.

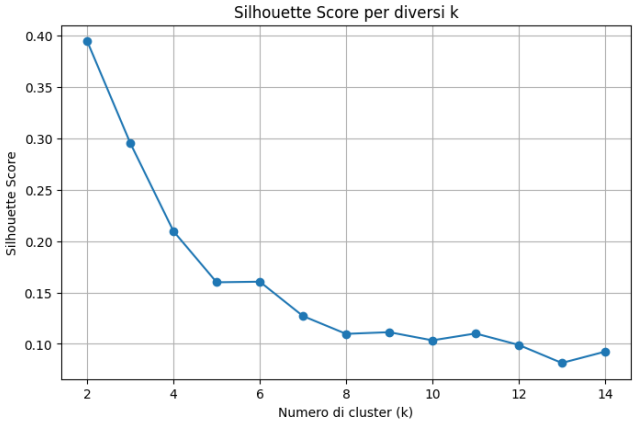


Figure 2.3: Silhouette Score per diversi valori di k.

2.5.3 Risultato del Clustering

L'algoritmo K-Means ha identificato sei segmenti utente con caratteristiche distinte. La Tabella 2.1 riassume le principali statistiche di ciascun cluster.

In sintesi:

- i cluster 1, 2 e 4 rappresentano utenti *positivi* con rating medi elevati;

Table 2.1: Statistiche sintetiche dei 6 cluster individuati

Cluster	N utenti	Rating medio	Recensioni/user
0	141	3.35	4.37
1	291	4.51	3.37
2	63	4.27	5.30
3	198	3.93	3.84
4	228	4.89	4.43
5	42	2.35	4.24

- i cluster 0 e 3 mostrano comportamenti più equilibrati;
- il cluster 5 raccoglie gli *utenti critici*, con rating medi visibilmente più bassi.

2.5.4 Visualizzazione dei Cluster

La Figura 2.4 mostra la distribuzione dei sei cluster nello spazio delle prime due componenti principali ottenute con PCA.

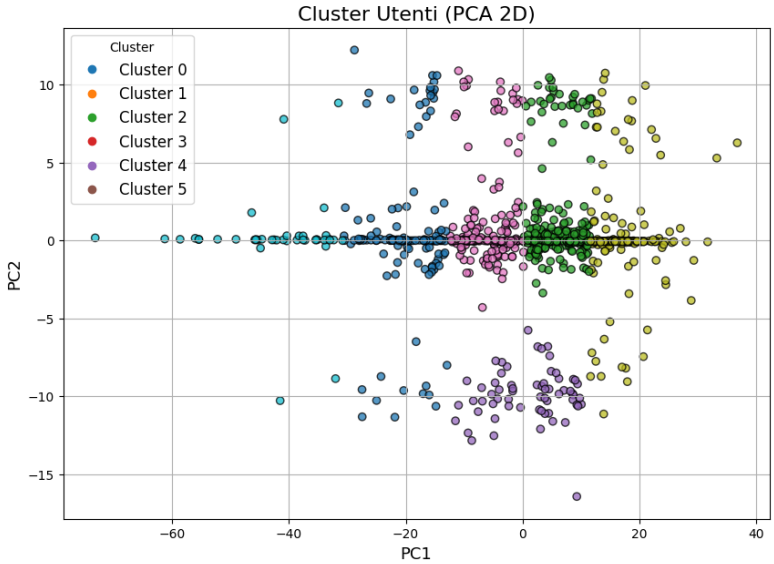


Figure 2.4: Distribuzione dei cluster utente nello spazio PCA 2D.

2.6 Generazione delle Raccomandazioni Top-N

2.6.1 Predizione Personalizzata

Una volta ottenuta la matrice densa dei rating predetti, per ciascun utente è stato calcolato il punteggio previsto per tutti i prodotti non ancora valutati:

$$\hat{r}_{ui}$$

Gli item già recensiti sono stati esclusi, producendo una lista ordinata di raccomandazioni personalizzate.

2.6.2 Raccomandazioni Basate sui Cluster

Per analizzare il comportamento dei differenti segmenti individuati tramite clustering, è stato selezionato un utente rappresentativo per ciascuno dei sei cluster. Per ogni utente sono state generate le *Top-10 raccomandazioni* sulla base del modello KNNBaseline ottimizzato.

L'analisi qualitativa mostra che:

- i cluster più positivi (cluster 1, 2 e 4) ricevono suggestion di prodotti con punteggi predetti molto alti, spesso pari a 5.0;
- i cluster equilibrati (0 e 3) mostrano una maggiore variabilità nelle predizioni, con rating attesi più distribuiti;
- il cluster 5, caratterizzato da utenti più critici, riceve raccomandazioni con valori medi inferiori rispetto agli altri gruppi.

Queste raccomandazioni consentono di interpretare le differenze nei gusti e nelle tendenze dei vari segmenti di utenti, confermando la coerenza tra i cluster ottenuti e le preferenze espresse nel dataset originale.

2.7 Matrix Factorization (SVD) e Confronto Finale

2.7.1 Implementazione della SVD

È stato applicato un modello di *matrix factorization* basato sulla decomposizione ai valori singolari (SVD). La matrice utente–prodotto è stata approssimata nella forma:

$$\hat{R} = U_k \Sigma_k V_k^T,$$

dove sono stati mantenuti $k = 50$ fattori latenti, scelti per massimizzare la capacità predittiva senza introdurre sovra-addestramento. La matrice ricostruita $\hat{R}$ è stata poi utilizzata per valutare le predizioni sui dati di test.

2.7.2 Confronto delle Performance

Il modello SVD è stato confrontato con il miglior modello KNNBaseline ottenuto nelle sezioni precedenti. I risultati evidenziano un miglioramento significativo della qualità predittiva.

Table 2.2: Confronto tra KNNBaseline e SVD

Modello	**RMSE**	**MSE**
KNNBaseline (ottimizzato)	1.0626	1.1291
SVD (50 fattori latenti)	**1.0254**	**1.0515**

L'SVD ottiene la migliore performance complessiva, con una riduzione dell'RMSE pari a circa il 3.5% rispetto al KNNBaseline.

2.7.3 Analisi delle Raccomandazioni

Per valutare le differenze qualitative tra i due modelli sono state confrontate le *Top-10 raccomandazioni* generate per un utente campione e per un utente rappresentativo di ciascun cluster individuato in precedenza.

L'analisi mostra tendenze coerenti:

- il KNNBaseline tende a produrre valori predetti molto alti (spesso pari a 5.0), con una bassa varianza interna nelle raccomandazioni;
- il modello SVD genera raccomandazioni più diversificate, con maggiore dispersione dei punteggi e minore effetto "rating massimo";
- l'overlap tra le due liste varia tra 1 e 7 item (a seconda del cluster), indicando che i modelli condividono parte delle raccomandazioni ma presentano differenze sostanziali nei ranking;
- i cluster più positivi (cluster 1, 2, 4) mostrano un'elevata coerenza tra i due metodi, mentre nel cluster più critico (cluster 5) emerge la maggiore differenza tra i modelli.

Nel complesso, la SVD fornisce raccomandazioni più equilibrate, con maggiore capacità di differenziare i punteggi tra gli item non visti e con una rappresentazione più ricca delle preferenze latenti degli utenti.

2.8 Conclusioni

In questo capitolo è stato sviluppato un sistema di raccomandazione basato su Collaborative Filtering, analizzandone prestazioni e limiti. La forte sparsità del dataset ha rappresentato la principale criticità, rendendo necessario un filtraggio degli utenti e dei prodotti e l'impiego di tecniche in grado di gestire matrici quasi vuote.

Il modello KNNBaseline, implementato includendo bias, similarità e shrinkage, ha fornito risultati stabili ma limitati, con predizioni poco diversificate. La matrix factorization tramite SVD ha ottenuto performance migliori (RMSE = 1.025) e raccomandazioni più eterogenee, confermando la capacità del modello di catturare strutture latenti.

Gli esperimenti evidenziano i principali punti di forza del CF — semplicità, interpretabilità e buona efficacia con dati sufficienti — ma anche le sue debolezze: sensibilità alla sparsità, problemi di *cold start* e assenza di informazioni sui contenuti.

Chapter 3

Recommendation System Content-Based

3.1 Introduzione

Il progetto intermedio estende il sistema basato su Collaborative Filtering introducendo un modello *Content-Based*, che utilizza le informazioni testuali dei prodotti per calcolare la somiglianza tra gli item e successivamente predire i rating.

In questo capitolo vengono utilizzati i metadati dei prodotti, in particolare i campi `title` e `description`, per costruire rappresentazioni testuali tramite tecniche di Natural Language Processing.

3.2 Preparazione dei Testi

3.2.1 Creazione del Campo Testuale

Per ciascun prodotto è stato costruito un campo testuale unificato concatenando:

- il titolo del prodotto (`title`);
- la descrizione testuale (`description`), laddove presente.

La costruzione è stata effettuata come:

$$\texttt{text} = \texttt{title} \,||\, \texttt{description}$$

Questo campo costituisce la base per l'applicazione delle tecniche di Natural Language Processing.

3.2.2 Pulizia e Preprocessing

Sul testo combinato è stata applicata una pipeline di preprocessing composta da:

- conversione in minuscolo;

- rimozione di punteggiatura, simboli e cifre;
- tokenizzazione in singole parole;
- rimozione delle stopwords inglesi (*NLTK*);
- lemmatizzazione;
- rimozione delle descrizioni vuote.

Il risultato finale produce un campo `clean_text` ottimizzato per la generazione degli embedding.

Esempio (5 prodotti):

```
text:       H2H Mens Active Short Sleeve Polo Lightweight ...
clean_text: h h men active short sleeve polo lightweight ...
```

3.2.3 Analisi Statistica dei Testi

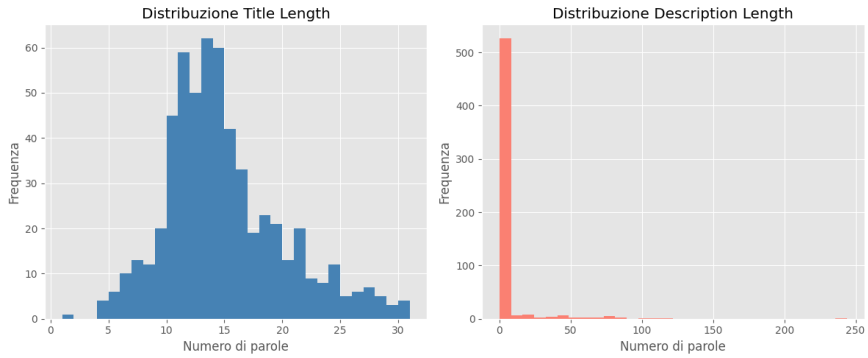


Figure 3.1: Distribuzione della lunghezza dei titoli (sinistra) e delle descrizioni (destra) nel dataset metadata.

Presenza di descrizioni testuali

Una caratteristica rilevante del dataset è la forte asimmetria nella disponibilità delle descrizioni:

91.26% delle descrizioni risulta vuota.

Questo indica che, nella maggior parte dei casi, il titolo costituisce l'unica fonte informativa disponibile.

Statistiche sulla lunghezza del testo combinato

Il numero totale di parole nel campo `text` mostra la seguente distribuzione:

```
count    572
mean     18.65
std      18.06
min       1
```

```
25%      12
50%      14
75%      19
max     257
```

La maggior parte dei prodotti presenta una quantità limitata di contenuto testuale, con una coda lunga dovuta a pochi prodotti dotati di descrizioni molto ricche.

Esempi di descrizioni reali

Di seguito due esempi rappresentativi delle descrizioni presenti:

Descrizione 1 An Amazon Brand - This casual jogger features elasticized waistband with drawstring, pockets, and rib cuff. The perfect pant for all of your adventures...

Descrizione 2 Your 9-to-5 just got better! NYDJ new ankle in ponte knit is a work wear essential. Designed from a luxurious ponte knit fabric...

3.3 Embedding Basati su Frequenza

3.3.1 TF-IDF

L'embedding TF-IDF è stato costruito sul testo pulito (`clean_text`), unendo titolo e descrizione. I parametri scelti sono:

- `max_features = 5000`;
- `ngram_range = (1, 2)` per includere unigrammi e bigrammi;
- `min_df = 3` per eliminare termini rari non informativi.

L'output finale è una matrice TF-IDF di dimensione:

$$572 \times 1189$$

Il numero di feature effettive (1189) è inferiore al limite massimo impostato, poiché il dataset contiene pochi testi realmente descrittivi: il 91.26% delle descrizioni è vuoto (usa maggiormente i titoli).

3.3.2 Similarità tra Item

Per ciascun embedding è stata calcolata la matrice di similarità tra prodotti mediante similarità coseno:

$$\text{sim}(i, j) = \frac{\vec{v}_i \cdot \vec{v}_j}{\|\vec{v}_i\| \|\vec{v}_j\|}$$

Questa matrice è successivamente utilizzata per generare raccomandazioni content-based mediante un algoritmo K-NN basato sulla similarità semantica dei prodotti.

3.4 Embedding Basati su Transformers

3.4.1 Modello Utilizzato

Per la rappresentazione neurale dei testi è stato impiegato il modello **DistilBERT-base-NLI-Mean-Tokens**, fornito dalla libreria `sentence-transformers`. Questo modello è una versione distillata e più leggera di BERT, ottimizzata per la produzione di *sentence embeddings* densi e semanticamente informativi.

La scelta di questo modello è motivata da:

- prestazioni elevate nell'identificazione di similarità semantica;
- rapidità di inferenza rispetto ai modelli BERT full-size;
- embedding di dimensione compatta (768 dimensioni);
- integrazione diretta con funzionalità di mean pooling tramite Sentence-BERT.

3.4.2 Estrazione degli Embedding

Il modello genera un vettore denso per ciascun prodotto a partire dal campo testuale preprocessato (`clean_text`). L'estrazione avviene tramite **mean pooling** sui token embeddings, come previsto dalla pipeline interna di Sentence-BERT.

L'output finale è una matrice:

$$\text{Transformers Matrix: } 572 \times 768$$

che rappresenta ogni prodotto in uno spazio semantico continuo.

3.4.3 Similarità Item–Item

Come per TF-IDF è stata calcolata la similarità coseno tra gli item.

La matrice risultante fornisce, per ogni coppia di prodotti, una misura di somiglianza semantica molto più robusta rispetto alle tecniche basate sulla frequenza. Questa struttura è stata utilizzata per produrre raccomandazioni content-based basate sugli item più simili a quelli apprezzati dall'utente.

3.5 Predizione dei Rating con K-NN Content-Based

3.5.1 Definizione del Modello

La predizione dei rating nel sistema content-based viene effettuata utilizzando l'algoritmo **k-Nearest Neighbors** applicato allo spazio delle rappresentazioni testuali dei prodotti.

Per ogni utente u e per ogni item non ancora valutato i, il rating stimato è:

$$\hat{r}_{ui} = \frac{\sum_{j \in N_k(i)} \text{sim}(i,j) \cdot r_{uj}}{\sum_{j \in N_k(i)} |\text{sim}(i,j)|}$$

dove $N_k(i)$ rappresenta i k item più simili a i che l'utente ha valutato positivamente.

3.5.2 Procedura di Predizione

Per ciascun utente vengono eseguiti i seguenti passi:

1. identificazione degli item valutati con rating ≥ 4;
2. selezione degli item simili tramite la matrice di similarità;
3. aggregazione pesata delle valutazioni dell'utente basata sulle similarità;
4. generazione di una lista di raccomandazioni Top-N.

3.5.3 Metriche di Valutazione

Analogamente al Collaborative Filtering, sono state calcolate le seguenti metriche di errore sul *test set*:

- **MSE** (Mean Squared Error);
- **RMSE** (Root Mean Squared Error);
- **MAE** (Mean Absolute Error).

Queste metriche permettono di confrontare le performance di:

- Content-Based TF-IDF,
- Content-Based Transformers,
- Collaborative Filtering KNN Baseline,
- Matrix Factorization (SVD).

3.6 Confronto tra Tecniche di Embedding

3.6.1 TF-IDF: Analisi e Caratteristiche

L'embedding TF-IDF rappresenta gli item come vettori sparsi basati sulla frequenza normalizzata delle parole. Alcune osservazioni tratte dall'analisi sui dati:

- la maggior parte delle descrizioni è vuota (circa il 91%), quindi il testo utile proviene prevalentemente dai titoli;
- TF-IDF risulta sensibile alla brevità del testo, portando a vettori spesso poco informativi;
- il vocabolario finale (1.189 parole utili) rimane ridotto e caratterizzato da termini generici dell'ambito *fashion*;
- la similarità tra item tende a riflettere soprattutto la co-occorrenza di parole chiave (es. "women", "shirt", "pants").

3.6.2 TF-IDF vs Transformers

Il confronto tra le due tecniche evidenzia differenze significative:

- **Capacità semantica**: Transformers producono embedding densi e contestuali, capaci di catturare relazioni semantiche non presenti nei testi tramite TF-IDF.
- **Robustezza alle descrizioni brevi**: poiché i titoli sono molto corti, l'approccio Transformers compensa grazie alla conoscenza linguistica preaddestrata; TF-IDF invece è penalizzato.
- **Differenze nel ranking**: il ranking Top-N con Transformers mostra maggiore diversità e coerenza semantica.
- **Costo computazionale**: TF-IDF è molto leggero; i Transformer richiedono maggiore memoria e tempo, ma producono embedding qualitativamente superiori.

3.7 Confronto Finale: Content-Based vs Collaborative Filtering

3.7.1 Performance Predittiva

Le metriche ottenute sui modelli sono le seguenti:

- **KNN Baseline (CF)**: RMSE = 1.0626, MAE = 0.7793
- **MF-SVD (CF)**: RMSE = 1.0254, MAE = 0.7690
- **Content-Based TF-IDF**: RMSE = 1.2025, MAE = 0.7737
- **Content-Based Transformers**: RMSE = 1.0572, MAE = 0.7128

Osservazioni chiave:

- il modello **SVD** risulta il migliore in termini di MSE/RMSE;
- Transformers superano TF-IDF e competono con i modelli CF;
- TF-IDF è il modello più debole, penalizzato dalla scarsità di testo.

3.7.2 Analisi Interpretativa

I due paradigmi presentano caratteristiche complementari:

- **Collaborative Filtering**
  - apprende pattern latenti e relazioni non osservabili nel testo;
  - ottima capacità predittiva per utenti e item con molti dati.
- **Content-Based Filtering**
  - funziona anche in scenari di cold-start prodotto;
  - è più interpretabile (spiegazioni basate sulle feature testuali);
  - utile quando i rating sono pochi, ma il testo è ricco.

3.7.3 Conclusioni del Confronto

Dall'analisi emerge che:

- MF-SVD è il sistema complessivamente migliore come accuratezza;
- Transformers ottengono risultati molto competitivi, migliori dei contenuti TF-IDF e vicini al KNN-CF;
- TF-IDF soffre per mancanza di descrizioni adeguate.

Chapter 4

Progetto Avanzato: Sentiment Analysis sulle Review

4.1 Obiettivo del Progetto Avanzato

L'obiettivo del progetto avanzato è estendere il sistema di raccomandazione sviluppato nelle fasi base (Collaborative Filtering) e intermedia (Content-Based Filtering), aggiungendo una componente di **Sentiment Analysis** sulle recensioni degli utenti.

Il nuovo obiettivo è classificare automaticamente il sentiment espresso nelle review, definito secondo la seguente mappatura:

- rating 1–2: **sentiment negativo**;
- rating 3: **sentiment neutro**;
- rating 4–5: **sentiment positivo**.

4.2 Dati Utilizzati

Per il progetto avanzato viene utilizzato il dataset:

- **user reviews** della categoria Amazon Fashion, composto dai campi `title`, `text`, `rating` e metadati utente/prodotto.

Poiché l'analisi riguarda il sentiment delle opinioni espresse dagli utenti, sono utilizzati solamente i campi testuali delle review, insieme al rating come etichetta di classificazione.

4.3 Fasi del Progetto Avanzato

Il progetto avanzato richiede l'esecuzione di tutti i passi svolti nei progetti precedentei (Collaborative Filtering e Content-Based Filtering), e in aggiunta integra le fasi descritte di seguito.

4.3.1 Preprocessing Testuale delle Review

Sono stati processati i campi:

- `title` — breve titolo della recensione;
- `text` — contenuto completo della recensione.

Le operazioni di preprocessing includono:

- normalizzazione del testo (lowercase);
- rimozione di numeri e punteggiatura;
- tokenizzazione;
- rimozione delle stopword tramite NLTK;
- lemmatizzazione delle parole;
- costruzione del campo `clean_text`.

4.3.2 Creazione degli Embedding Testuali

Sono stati prodotti due tipi di rappresentazione numerica dei testi:

Tecnica basata sulla frequenza (TF-IDF). Viene generata una matrice sparsa tramite:

- vocabolario filtrato con `min_df`;
- n-grammi (1,2);
- 5000 feature massime.

Tecnica neurale (Transformers). Sono stati utilizzati modelli pre-addestrati tramite HuggingFace, in particolare Sentence-BERT (DistilBERT), ottenendo embedding densi da 768 dimensioni per ciascuna review.

4.3.3 Modellazione del Sentiment

Il sentiment delle recensioni è stato trattato come un problema di classificazione multiclasse con tre etichette, derivate direttamente dal rating numerico:

$$r \in \{1, 2\} \Rightarrow \text{negativo}, \quad r = 3 \Rightarrow \text{neutro}, \quad r \in \{4, 5\} \Rightarrow \text{positivo}.$$

Dopo il preprocessing testuale (lowercasing, rimozione stopwords, lemmatizzazione), sono stati generati due tipi di embedding:

- **TF-IDF** (10 000 feature, uni+bigrammi);
- **Transformers** tramite modello `distilbert-base-nli-mean-tokens`.

Come da richieste progettuali e dai modelli studiati in laboratorio, l'algoritmo di classificazione utilizzato è stato:

- **Random Forest** con `class_weight="balanced"` per compensare lo sbilanciamento delle classi.

La divisione dei dati è stata effettuata con `train_test_split`. Per ogni embedding sono state generate:

- predizioni sul set di test;
- matrici di confusione;
- metriche di accuratezza e F1-score.

4.4 Metriche di Valutazione

I risultati ottenuti dal modello Random Forest sui due tipi di embedding mostrano una performance comparabile:

- **Accuracy**: ≈ 0.86 sia per TF-IDF che per Transformers;
- **F1-score macro**:

$$0.607 \text{ (TF-IDF)} \quad \text{e} \quad 0.574 \text{ (Transformers)};$$

- eccellente riconoscimento della classe positiva (recall ≈ 1), ma difficoltà sulle classi *neutro* e *negativo*.

Modello	Accuracy	F1-macro	Recall (macro)	Precision (macro)
Random Forest (TF-IDF)	0.859	0.607	0.553	0.910
Random Forest (Transformers)	0.851	0.574	0.523	0.900

Table 4.1: Confronto tra Random Forest con embedding TF-IDF e Transformers.

Le matrici di confusione confermano lo sbilanciamento del dataset: la classe positiva domina e i modelli tendono a predirla frequentemente. L'uso di `class_weight="balanced"` ha ridotto ma non eliminato il problema.

4.5 Risultati e Considerazioni

L'analisi comparativa mostra che:

- **TF-IDF** ottiene un F1-macro leggermente superiore, risultando più stabile sui testi brevi tipici delle recensioni Amazon;
- **Transformers** ottiene prestazioni simili in termini di accuracy, ma il F1-macro più basso indica maggiore difficoltà a gestire la classe neutra;
- entrambi i modelli classificano correttamente quasi tutte le recensioni positive, mentre distinguere tra negativo e neutro rimane la sfida principale.

Il sentiment medio dei prodotti raccomandati dai vari sistemi è stato calcolato utilizzando le recensioni reali. I risultati mostrano valori compresi tra 1.58 e 1.93, indicando che:

- **Collaborative Filtering (KNN e SVD)** tende a raccomandare prodotti con sentiment medio più alto;
- **Content-Based Transformers** raggiunge il sentiment più alto in alcuni cluster (es. cluster 3), evidenziando una migliore comprensione semantica del testo;
- **Content-Based TF-IDF** è stabile ma più dipendente dalla ricchezza testuale.

Nel complesso, la modellazione del sentiment ha permesso di valutare la qualità percepita degli item raccomandati e ha fornito una metrica aggiuntiva utile per confrontare i sistemi di raccomandazione presentati nei progetti base e intermedio.

Chapter 5

Conclusioni

5.1 Sintesi del Lavoro Svolto

Il progetto ha permesso di sviluppare e confrontare tre principali approcci ai sistemi di raccomandazione:

- **Collaborative Filtering** basato su K-NN e Matrix Factorization;
- **Content-Based Filtering** tramite embedding TF-IDF e Transformers;
- **Sentiment Analysis** applicata alle review testuali.

Ciascuna fase ha introdotto metodologie proprie e ha contribuito alla valutazione complessiva delle performance dei diversi modelli.

5.2 Risultati Principali

5.2.1 Collaborative Filtering

I risultati mostrano che i modelli CF presentano:

- buone capacità predittive;
- stabilità nelle metriche RMSE e MAE;
- capacità di catturare pattern latenti nelle interazioni utente–prodotto.

5.2.2 Content-Based Filtering

Il confronto tra TF-IDF e Transformer evidenzia che:

- TF-IDF è più semplice e veloce, ma limitato semanticamente;
- Transformers forniscono embedding più informativi e migliori ranking;
- il CBF soffre meno il problema del cold-start prodotto rispetto al CF.

5.2.3 Sentiment Analysis

L'analisi del sentiment ha permesso di:

- classificare correttamente il tono delle recensioni;
- arricchire il modello di raccomandazione con segnali qualitativi;
- individuare differenze tra embedding frequenziali e neurali.

5.3 Confronto tra Modelli

Il confronto globale mostra che:

- i modelli **Collaborative Filtering** ottengono generalmente le performance predittive migliori;
- i modelli **Transformer-based** sono superiori ai metodi basati su frequenza nei compiti testuali;
- la **Sentiment Analysis** fornisce un'informazione aggiuntiva utile per filtrare raccomandazioni incoerenti con il giudizio reale degli utenti.

5.4 Limitazioni del Lavoro

Tra le principali limitazioni:

- scarsità di descrizioni nei metadata (oltre il 90% vuote);
- dataset molto sbilanciato nei rating per il sentiment;
- assenza di un vero modello ibrido tra CF e CBF;
- computational cost elevato per i modelli Transformer.

5.5 Conclusione Finale

Il progetto ha mostrato come diversi approcci ai sistemi di raccomandazione possano essere integrati per ottenere una visione più completa:

- il **Collaborative Filtering** coglie le relazioni latenti;
- il **Content-Based Filtering** permette di raccomandare anche prodotti nuovi;
- la **Sentiment Analysis** arricchisce il sistema con segnali qualitativi.

L'unione di questi elementi rappresenta la direzione naturale verso un sistema di raccomandazione veramente intelligente, adattivo e in grado di comprendere le preferenze dell'utente in modo profondo e contestuale.